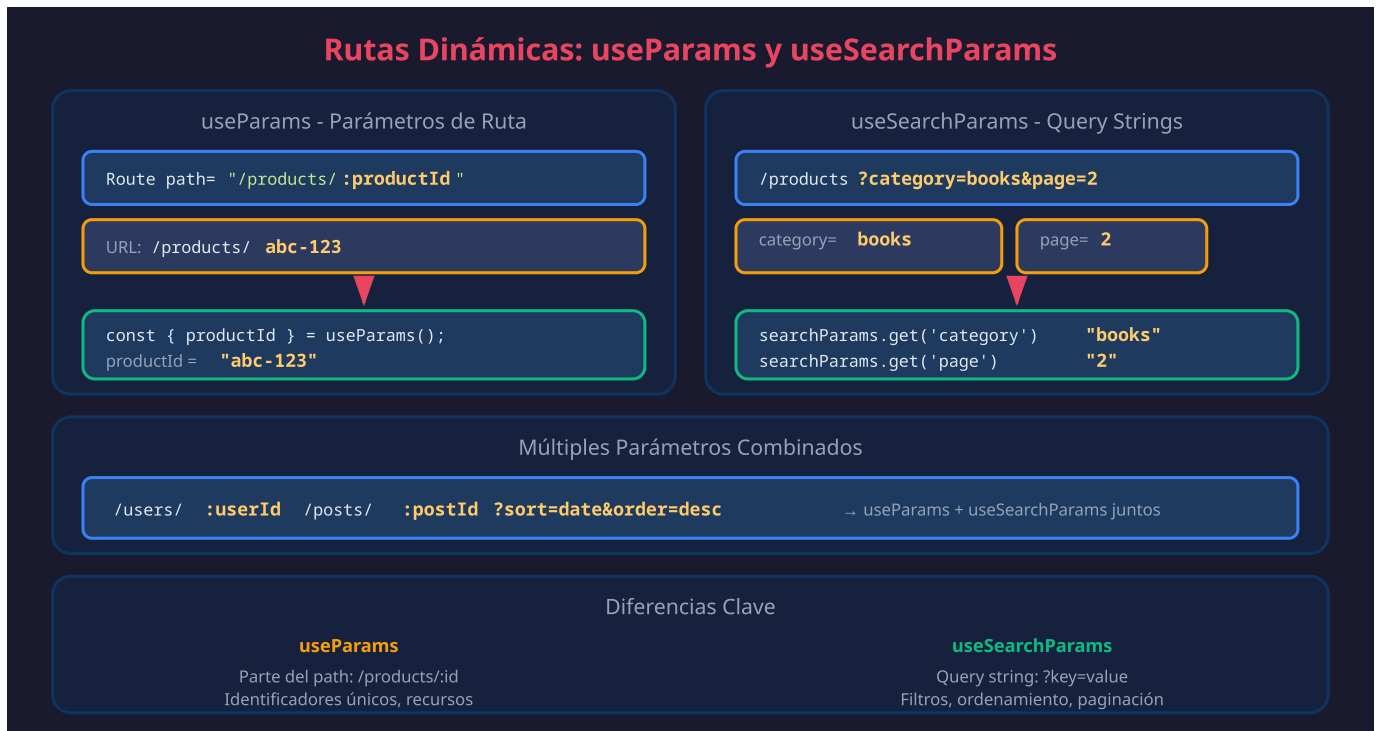


Rutas Dinámicas y Parámetros

Objetivos

- Implementar rutas con parámetros dinámicos
- Usar useParams con TypeScript
- Manejar query strings con useSearchParams
- Implementar navegación programática

1. Rutas con Parámetros



Sintaxis de Parámetros

```
// =====  
// QUÉ: Rutas con segmentos dinámicos usando :param  
// PARA: Crear páginas que varían según un identificador  
// IMPACTO: Una sola definición de ruta para múltiples recursos  
// =====  
  
import { Routes, Route } from 'react-router-dom';  
  
const App: React.FC = () => {  
  return (  
    <Routes>  
      {/* Rutas estáticas */}  
      <Route  
        path="/"  
        element={<HomePage />}  
      />  
      <Route  
        path="/products"
```

```

    element={<ProductListPage />}
  />

  { /* Ruta dinámica con un parámetro */}
  <Route
    path="/products/:productId"
    element={<ProductDetailPage />}
  />

  { /* Ruta con múltiples parámetros */}
  <Route
    path="/users/:userId/posts/:postId"
    element={<UserPostPage />}
  />

  { /* Parámetro opcional (con ?) */}
  <Route
    path="/search/:query?"
    element={<SearchPage />}
  />
</Routes>
);
};

// URLs que matchean:
// /products/123      → productId = "123"
// /products/abc-def  → productId = "abc-def"
// /users/5/posts/10  → userId = "5", postId = "10"
// /search            → query = undefined
// /search/react       → query = "react"

```

2. useParams Hook

Uso Básico

```

// =====
// QUÉ: Hook para acceder a parámetros de la URL
// PARA: Obtener valores dinámicos de la ruta actual
// IMPACTO: Componentes que reaccionan al parámetro de URL
// =====

import { useParams } from 'react-router-dom';

// Los parámetros siempre son string | undefined
const ProductDetailPage: React.FC = () => {
  const { productId } = useParams();

  // productId es string | undefined
  console.log(productId); // "123" si URL es /products/123

  return (
    <div>
      <h1>Producto: {productId}</h1>
    </div>
  );
};

```

```
);  
};
```

Tipado con TypeScript

```
// =====  
// QUÉ: Tipar parámetros de URL con TypeScript  
// PARA: Obtener autocompletado y validación en tiempo de compilación  
// IMPACTO: Código más seguro, menos errores de runtime  
// =====  
  
import { useParams } from 'react-router-dom';  
  
// Método 1: Tipo inline  
const ProductDetailPage: React.FC = () => {  
  const { productId } = useParams<{ productId: string }>();  
  // productId es string | undefined  
  
  if (!productId) {  
    return <div>Producto no encontrado</div>;  
  }  
  
  return <div>Producto: {productId}</div>;  
};  
  
// Método 2: Interface separada  
interface ProductParams {  
  productId: string;  
}  
  
const ProductDetailPageTyped: React.FC = () => {  
  const { productId } = useParams<ProductParams>();  
  
  return <div>Producto: {productId}</div>;  
};  
  
// Método 3: Genérico con Record  
const UserPostPage: React.FC = () => {  
  const params = useParams<Record<'userId' | 'postId', string>>();  
  
  return (  
    <div>  
      <p>Usuario: {params.userId}</p>  
      <p>Post: {params.postId}</p>  
    </div>  
  );  
};
```

Validación de Parámetros

```
// =====  
// QUÉ: Validar y convertir parámetros de URL  
// PARA: Asegurar que los parámetros son válidos antes de usar  
// IMPACTO: Prevenir errores con IDs inválidos
```

```
// =====

import { useParams, Navigate } from 'react-router-dom';
import { useEffect, useState } from 'react';

interface Product {
  id: number;
  name: string;
  price: number;
}

const ProductDetailPage: React.FC = () => {
  const { productId } = useParams<{ productId: string }>();
  const [product, setProduct] = useState<Product | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  // Validar que productId existe y es un número
  const numericId = productId ? parseInt(productId, 10) : NaN;

  if (!productId || isNaN(numericId)) {
    return (
      <Navigate
        to="/products"
        replace
      />
    );
  }

  useEffect(() => {
    const fetchProduct = async () => {
      try {
        setLoading(true);
        const response = await fetch(`/api/products/${numericId}`);

        if (!response.ok) {
          throw new Error('Producto no encontrado');
        }

        const data = await response.json();
        setProduct(data);
      } catch (err) {
        setError(err instanceof Error ? err.message : 'Error desconocido');
      } finally {
        setLoading(false);
      }
    };

    fetchProduct();
  }, [numericId]);

  if (loading) return <p>Cargando...</p>;
  if (error) return <p>Error: {error}</p>;
  if (!product) return <p>Producto no encontrado</p>;

  return (

```

```

    <div>
      <h1>{product.name}</h1>
      <p>Precio: ${product.price}</p>
    </div>
  );
};

```

3. useSearchParams Hook

Query Strings

```

// =====
// QUÉ: Hook para manejar query strings (?key=value)
// PARA: Filtros, búsquedas, paginación en la URL
// IMPACTO: Estado compartible vía URL, SEO friendly
// =====

import { useSearchParams } from 'react-router-dom';

// URL: /products?category=electronics&sort=price&page=2
const ProductListPage: React.FC = () => {
  const [searchParams, setSearchParams] = useSearchParams();

  // Leer parámetros
  const category = searchParams.get('category'); // "electronics" | null
  const sort = searchParams.get('sort'); // "price" | null
  const page = searchParams.get('page'); // "2" | null

  // Obtener todos los valores de un parámetro con múltiples valores
  // URL: /products?tag=new&tag=sale
  const tags = searchParams.getAll('tag'); // ["new", "sale"]

  // Verificar si existe un parámetro
  const hasCategory = searchParams.has('category'); // true

  return (
    <div>
      <p>Categoría: {category ?? 'Todas'}</p>
      <p>Ordenar por: {sort ?? 'Relevancia'}</p>
      <p>Página: {page ?? '1'}</p>
    </div>
  );
};

```

Modificar Query Strings

```

// =====
// QUÉ: Actualizar query strings de forma reactiva
// PARA: Filtros interactivos que se reflejan en la URL
// IMPACTO: URL compartible, estado persistente al recargar
// =====

import { useSearchParams } from 'react-router-dom';

```

```

const ProductFilters: React.FC = () => {
  const [searchParams, setSearchParams] = useSearchParams();

  const currentCategory = searchParams.get('category') ?? '';
  const currentSort = searchParams.get('sort') ?? 'relevance';

  // Actualizar un parámetro (reemplaza todos)
  const handleCategoryChange = (category: string) => {
    setSearchParams({ category, sort: currentSort });
  };

  // Actualizar preservando otros parámetros
  const handleSortChange = (sort: string) => {
    setSearchParams((prev) => {
      prev.set('sort', sort);
      return prev;
    });
  };

  // Eliminar un parámetro
  const clearCategory = () => {
    setSearchParams((prev) => {
      prev.delete('category');
      return prev;
    });
  };

  // Limpiar todos los filtros
  const clearAll = () => {
    setSearchParams({});
  };

  return (
    <div className="filters">
      <select
        value={currentCategory}
        onChange={(e) => handleCategoryChange(e.target.value)}>
        <option value="">Todas las categorías</option>
        <option value="electronics">Electrónica</option>
        <option value="clothing">Ropa</option>
        <option value="books">Libros</option>
      </select>

      <select
        value={currentSort}
        onChange={(e) => handleSortChange(e.target.value)}>
        <option value="relevance">Relevancia</option>
        <option value="price">Precio</option>
        <option value="name">Nombre</option>
      </select>

      <button onClick={clearAll}>Limpiar filtros</button>
    </div>
  );
};

```

Paginación con searchParams

```
// =====
// QUÉ: Implementar paginación usando query strings
// PARA: Navegación entre páginas de resultados
// IMPACTO: Página actual persistente en URL
// =====

import { useSearchParams } from 'react-router-dom';

interface PaginationProps {
  totalPages: number;
}

const Pagination: React.FC<PaginationProps> = ({ totalPages }) => {
  const [searchParams, setSearchParams] = useSearchParams();

  const currentPage = parseInt(searchParams.get('page') ?? '1', 10);

  const goToPage = (page: number) => {
    setSearchParams((prev) => {
      if (page === 1) {
        prev.delete('page'); // No mostrar page=1 en URL
      } else {
        prev.set('page', String(page));
      }
      return prev;
    });
  };

  return (
    <div className="pagination">
      <button
        onClick={() => goToPage(currentPage - 1)}
        disabled={currentPage <= 1}>
        Anterior
      </button>

      <span>
        Página {currentPage} de {totalPages}
      </span>

      <button
        onClick={() => goToPage(currentPage + 1)}
        disabled={currentPage >= totalPages}>
        Siguiente
      </button>
    </div>
  );
};
```

4. useNavigate Hook

Navegación Programática

```
// =====
// QUÉ: Hook para navegar programáticamente
// PARA: Redirigir después de acciones (submit, login, etc.)
// IMPACTO: Control total sobre cuándo y cómo navegar
// =====

import { useNavigate } from 'react-router-dom';

const LoginForm: React.FC = () => {
  const navigate = useNavigate();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    try {
      await loginUser(/* credentials */);

      // Navegar a dashboard después del login
      navigate('/dashboard');
    } catch (error) {
      console.error('Login failed');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      {/* campos del formulario */}
      <button type="submit">Iniciar Sesión</button>
    </form>
  );
};
```

Opciones de useNavigate

```
// =====
// QUÉ: Opciones avanzadas de navegación
// PARA: Controlar historial, pasar estado, etc.
// IMPACTO: Navegación flexible según el caso de uso
// =====

import { useNavigate } from 'react-router-dom';

const NavigationExamples: React.FC = () => {
  const navigate = useNavigate();

  // Navegación básica
  const goToHome = () => navigate('/');

  // Navegación con replace (no agrega al historial)
  const goToLoginReplace = () => navigate('/login', { replace: true });

  // Navegación con estado
  const goToCheckout = () => {
    navigate('/checkout', {
```

```

    state: { fromCart: true, itemCount: 5 },
  });
};

// Navegación relativa
const goBack = () => navigate(-1); // Equivale a history.back()
const goForward = () => navigate(1); // Equivale a history.forward()
const goBackTwo = () => navigate(-2);

// Navegación a ruta relativa al actual
const goToChild = () => navigate('child-route'); // Relativa
const goToSibling = () => navigate('../sibling'); // Subir y bajar

return (
  <div>
    <button onClick={goToHome}>Ir a Home</button>
    <button onClick={goBack}>- Atrás</button>
    <button onClick={goForward}>Adelante →</button>
    <button onClick={goToCheckout}>Ir a Checkout</button>
  </div>
);
};

```

Acceder al Estado de Navegación

```

// =====
// QUÉ: Leer estado pasado durante la navegación
// PARA: Pasar datos entre páginas sin URL
// IMPACTO: Datos sensibles que no deben estar en URL
// =====

import { useLocation, useNavigate } from 'react-router-dom';

// Página origen: pasa estado
const CartPage: React.FC = () => {
  const navigate = useNavigate();

  const proceedToCheckout = () => {
    navigate('/checkout', {
      state: {
        items: cartItems,
        total: calculateTotal(),
        couponApplied: true,
      },
    });
  };

  return <button onClick={proceedToCheckout}>Proceder al pago</button>;
};

// Página destino: lee estado
interface CheckoutState {
  items: CartItem[];
  total: number;
  couponApplied: boolean;
}

```

```

}

const CheckoutPage: React.FC = () => {
  const location = useLocation();
  const navigate = useNavigate();

  // Tipar el estado
  const state = location.state as CheckoutState | null;

  // Validar que tenemos estado (usuario llegó desde el carrito)
  if (!state?.items) {
    // Redirigir si acceden directamente
    return (
      <Navigate
        to="/cart"
        replace
      />
    );
  }

  return (
    <div>
      <h1>Checkout</h1>
      <p>Total: ${state.total}</p>
      <p>Artículos: {state.items.length}</p>
      {state.couponApplied && <p>✓ Cupón aplicado</p>}
    </div>
  );
};

```

5. useLocation Hook

```

// =====
// QUÉ: Hook para obtener información de la URL actual
// PARA: Acceder a pathname, search, hash y state
// IMPACTO: Componentes conscientes de su ubicación
// =====

import { useLocation } from 'react-router-dom';

const LocationDebug: React.FC = () => {
  const location = useLocation();

  // location contiene:
  // {
  //   pathname: "/products/123",
  //   search: "?color=blue",
  //   hash: "#reviews",
  //   state: { fromHome: true },
  //   key: "default"
  // }

  return (
    <div>

```

```

    <p>Pathname: {location.pathname}</p>
    <p>Search: {location.search}</p>
    <p>Hash: {location.hash}</p>
    <p>State: {JSON.stringify(location.state)}</p>
  </div>
);
};

// Caso de uso: Analytics
const usePageTracking = () => {
  const location = useLocation();

  useEffect(() => {
    // Enviar pageview a analytics
    analytics.pageview(location.pathname + location.search);
  }, [location]);
};

```

6. Ejemplo Completo: Lista y Detalle

```

// =====
// QUÉ: Patrón maestro-detalle con rutas dinámicas
// PARA: Listar items y ver detalles de cada uno
// IMPACTO: Navegación natural entre lista y detalle
// =====

// src/App.tsx
import { Routes, Route } from 'react-router-dom';
import { ProductListPage } from '../pages/ProductListPage';
import { ProductDetailPage } from '../pages/ProductDetailPage';

const App: React.FC = () => (
  <Routes>
    <Route
      path="/products"
      element={<ProductListPage />}
    />
    <Route
      path="/products/:productId"
      element={<ProductDetailPage />}
    />
  </Routes>
);

// src/pages/ProductListPage.tsx
import { Link, useSearchParams } from 'react-router-dom';

interface Product {
  id: number;
  name: string;
  category: string;
}

const products: Product[] = [

```

```

    { id: 1, name: 'Laptop', category: 'electronics' },
    { id: 2, name: 'Camiseta', category: 'clothing' },
    { id: 3, name: 'React Book', category: 'books' },
  ];

const ProductListPage: React.FC = () => {
  const [searchParams, setSearchParams] = useSearchParams();
  const categoryFilter = searchParams.get('category');

  const filteredProducts = categoryFilter
    ? products.filter((p) => p.category === categoryFilter)
    : products;

  return (
    <div>
      <h1>Productos</h1>

      {/* Filtros */}
      <div>
        <button onClick={() => setSearchParams({})}>Todos</button>
        <button onClick={() => setSearchParams({ category: 'electronics' })}>
          Electrónica
        </button>
        <button onClick={() => setSearchParams({ category: 'clothing' })}>
          Ropa
        </button>
      </div>

      {/* Lista */}
      <ul>
        {filteredProducts.map((product) => (
          <li key={product.id}>
            <Link to={` /products/${product.id}`}>{product.name}</Link>
          </li>
        ))}
      </ul>
    </div>
  );
};

```

```

// src/pages/ProductDetailPage.tsx
import { useParams, Link, useNavigate } from 'react-router-dom';

const ProductDetailPage: React.FC = () => {
  const { productId } = useParams<{ productId: string }>();
  const navigate = useNavigate();

  const product = products.find((p) => p.id === Number(productId));

  if (!product) {
    return (
      <div>
        <h1>Producto no encontrado</h1>
        <button onClick={() => navigate('/products')}>
          Volver a productos
        </button>
      </div>
    );
  }
};

```

```
        </div>
    );
}

return (
    <div>
        <Link to="/products">← Volver</Link>
        <h1>{product.name}</h1>
        <p>Categoría: {product.category}</p>
        <p>ID: {product.id}</p>
    </div>
);
};
```



Recursos Adicionales

- [useParams API](#)
- [useSearchParams API](#)
- [useNavigate API](#)



Checklist de Comprensión

- ☐ Sé definir rutas con parámetros dinámicos (:param)
- ☐ Puedo tipar useParams correctamente
- ☐ Entiendo cómo usar useSearchParams para filtros
- ☐ Sé navegar programáticamente con useNavigate
- ☐ Puedo pasar y leer estado entre navegaciones

Siguiente: [04-navegacion-avanzada.md](#)